

C++ Classes, Headers, and Object Lifetime

Merged sheet based on ECM2433 L12 Classes and L13 Memory, Pointers, and References. It combines access control, headers, constructors, destructors, copying, member initializer lists, and g++ compilation.

What You Need To Know

1. Access control and encapsulation

private members are only accessible inside the class. protected members are also accessible inside derived classes. public members are available to outside code. A class defaults to private; a struct defaults to public.

```
class Account {
    double balance;
public:
    double get_balance() const { return balance; }
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
};
```

2. this and member initializer lists

this is a pointer to the current object. Use it when a parameter shadows a member name. A member initializer list initializes members and base classes before the constructor body runs.

```
class Box {
    int width;
public:
    Box(int width) : width(width) {}
    int get_width() const { return width; }
};

class TemperatureSensor : public Sensor {
    double celsius;
public:
    TemperatureSensor(int id, const std::string& name, double celsius)
        : Sensor(id, name), celsius(celsius) {}
};
```

3. Headers and source files

A header declares the class. A .cpp file defines member functions using the scope-resolution prefix, such as Counter::increment. Include guards prevent the same declarations being processed twice.

```
/* counter.h */
#ifndef COUNTER_H
#define COUNTER_H

class Counter {
    int count;
public:
    Counter();
    void increment();
    int get_count() const;
};

#endif

/* counter.cpp */
#include "counter.h"

Counter::Counter() : count(0) {}
void Counter::increment() { count++; }
int Counter::get_count() const { return count; }
```

4. Constructors, copy constructors, and destructors

A constructor initializes an object. A copy constructor builds a new object from an existing object. A destructor runs when an object is destroyed. If a class owns a raw resource, copying and destruction must be designed carefully; otherwise prefer RAII types such as std::string or smart pointers.

```
class Trace {
public:
    Trace() { std::cout << "ctor\n"; }
    Trace(const Trace&) { std::cout << "copy\n"; }
    ~Trace() { std::cout << "dtor\n"; }
};
```

5. Compiling multiple C++ files

```
g++ -std=c++17 -c counter.cpp -o counter.o
g++ -std=c++17 -c main.cpp -o main.o
g++ main.o counter.o -o app
```

A pointer to a class member uses the class in the pointer type, for example `int Sensor::* field = &Sensor::id;`. For an object `s`, access it with `s.*field`.

Practice Questions

- 1 Write a guarded `counter.h` declaration for a `Counter` class with private `int count`, a default constructor, `increment`, and `get_count`.
- 2 In `counter.cpp`, define `increment`. Make sure you use the required class prefix.

Question 3 - access control:

```
class Base {
private:
    int x;
protected:
    int y;
public:
    int z;
};
```

- 3 Which of `x`, `y`, and `z` can outside code access? Which can a derived class access?

Question 4 - complete the setter:

```
class User {
    int age;
public:
    void set_age(int age) {
        /* reject negative ages, otherwise store the age */
    }
};
```

- 4 Complete `set_age`. Use `this` if needed.

Question 5 - fix the constructor bug:

```
class Box {
    int width;
public:
    Box(int width) {
        width = width;
    }
};
```

- 5 Explain the bug and fix it using either `this` or an initializer list.
- 6 Write a constructor for `TemperatureSensor` that initializes the base `Sensor(id, name)` and member `celsius`.

Question 7 - trace object lifetime:

```
int main() {
    Trace a;
    Trace b = a;
}
```

- 7 Identify where construction, copy construction, and destruction happen in `main`.

Question 8 - copy constructor signature:

```
class Buffer {
public:
    Buffer(const Buffer other) {}
};
```

- 8 Fix the copy constructor parameter type and explain why it should be a `const` reference.
- 9 A class only contains `std::string name;`. Is `~Thing() = default;` normally acceptable? Explain.
- 10 Write the three `g++` commands to compile `counter.cpp` and `main.cpp` separately, then link them into `app`.

Mark Scheme

Attempt all questions before checking.

- 1 Use `#ifndef COUNTER_H, #define COUNTER_H`, the class declaration, and `#endif`. Members before `public:` are private.
- 2 `void Counter::increment() { count++; }`. The `Counter::` prefix says this is the class member function.
- 3 Outside code can access only `z`. A derived class can access `y` and `z`, but not private `x`.
- 4 Expected: `if (age < 0) return; this->age = age;`
- 5 `width = width` assigns the parameter to itself. Fix with `Box(int width) : width(width) {}` or `this->width = width;`.
- 6 `TemperatureSensor(int id, const std::string& name, double celsius) : Sensor(id, name), celsius(celsius) {}`
- 7 Trace `a;` calls the default constructor. Trace `b = a;` calls the copy constructor. Destructors run for `b` then `a` at the end of `main`.
- 8 Use `Buffer(const Buffer& other)`. Passing by value would require copying before entering the copy constructor, causing recursion and unnecessary copying.
- 9 Yes, normally. `std::string` cleans itself up through RAII, so a default destructor is enough unless the class owns some manual resource.
- 10 `g++ -std=c++17 -c counter.cpp -o counter.o; g++ -std=c++17 -c main.cpp -o main.o; g++ main.o counter.o -o app.`